BLACKOUT

BLACKOUT Security Lab

LLM 개발자를 위한 Web & 인프라 마스터 클래스 (Full-Stack & Security)

트래픽의 흐름부터 코드 레벨의 취약점, 그리고 이를 격리하고 배포하는 인프라까지 아우르는 정예 마케팅/개발 가이드입니다.

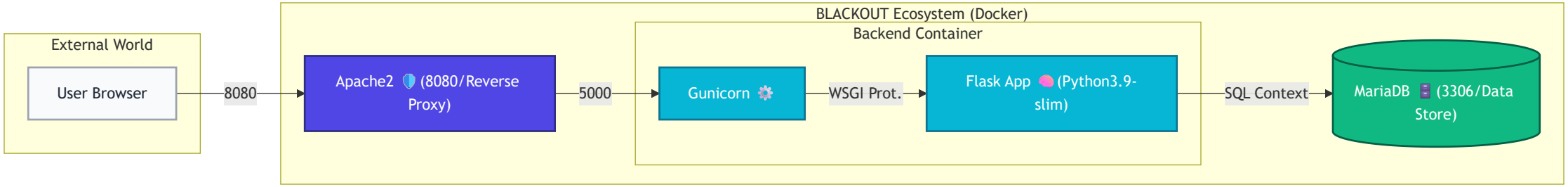
1. 프로젝트 비전: 왜 이 교육을 하는가?

- LLM 개발은 모델링만큼이나 이를 지탱하는 **안전한 그릇(Infrastructure)**이 중요합니다.
- 단순한 '장난감' 프로젝트가 아닌, 상용 수준의 아키텍처를 직접 구축하며 Full-Stack의 감각을 익힙니다.
- **Goal:** 구조적 사고력을 갖춘 엔지니어로의 도약.

2. 통합 아키텍처 (Big Picture)

- 마이크로 서비스의 핵심인 '격리'와 '통합'을 동시에 구현하였습니다.

[Diagram] 통합 아키텍처 조감도



3. 핵심 기술 스택 (Technology Stack)

- **Language:** Python 3.9 (Stable & Lightweight)
- **Framework:** Flask (Flexible Micro-framework)
- **Frontend:** HTML5, CSS3, Bootstrap 5, Jinja2
- **Database:** MariaDB 10.11 (Production-grade RDBMS)
- **Infrastructure:** Docker, Docker Compose, Apache HTTPD
- **Security:** OAuth 2.0 (Google/OAuthlib), MD5 (Vulnerable-by-design)

4. 트래픽 시나리오 (Traffic Flow)

1. **Request:** 유저가 8080 포트로 접속하면 Apache가 주문을 받습니다.
2. **Reverse Proxy:** Apache가 내부망의 5000 포트(Gunicorn)로 주문을 토스합니다.
3. **Execution:** Flask가 비즈니스 로직(DB 조회, 연산)을 수행합니다.
4. **Response:** 완성된 HTML을 역순으로 유저에게 응답합니다.

5. Part 2: 개발 환경 및 도구 (Dev Tooling)

- "좋은 목수는 연장을 탓하지 않지만, 고수는 최상의 연장을 세팅합니다."
- 효율적인 개발을 위한 환경 설정(VS Code / Python / Git)에 대해 알아봅니다.

6. VS Code: Modern IDE Configuration

- **Marp:** 현재 보고 계신 이 PT를 Markdown으로 실시간 미리보기 하며 편집하는 도구.
- **Mermaid Preview:** 복잡한 다이어그램을 코드로 즉시 시각화합니다.
- **Docker Extension:** 컨테이너의 상태와 로그를 GUI로 실시간 모니터링합니다.
- **SQLite Viewer:** 로컬 테스트 DB를 즉시 열어보는 유틸리티.

7. Python & Pip: Dependency Management

- **Python 3.9-slim:** 뚱뚱한 표준 이미지 대신, 꼭 필요한 환경만 담았습니다.
- **requirements.txt:** 모든 라이브러리를 버전까지 고정하여 어디서든 동일하게 작동하게 합니다.
- **Virtualenv:** 내 컴퓨터의 다른 프로젝트와 라이브러리가 꼬이지 않게 '나만의 방'을 만듭니다.

8. Git & Terminal Workflow

- **Git:** 코드의 '타임머신'. 실수해도 언제든 과거로 돌아갈 수 있습니다.
- **PowerShell / Bash:** GUI보다 강력한 커맨드로 패키지 설치와 컨테이너 제어를 수행합니다.
- **.env:** 비밀번호, API 키 등 민감 정보를 코드와 분리해 관리합니다.

9. Part 3: 웹 프레임워크 Deep-Dive

- 뼈대(HTML)와 살(CSS), 그리고 피(Logic)가 흐르는 웹 애플리케이션의 내부를 들여다봅니다.

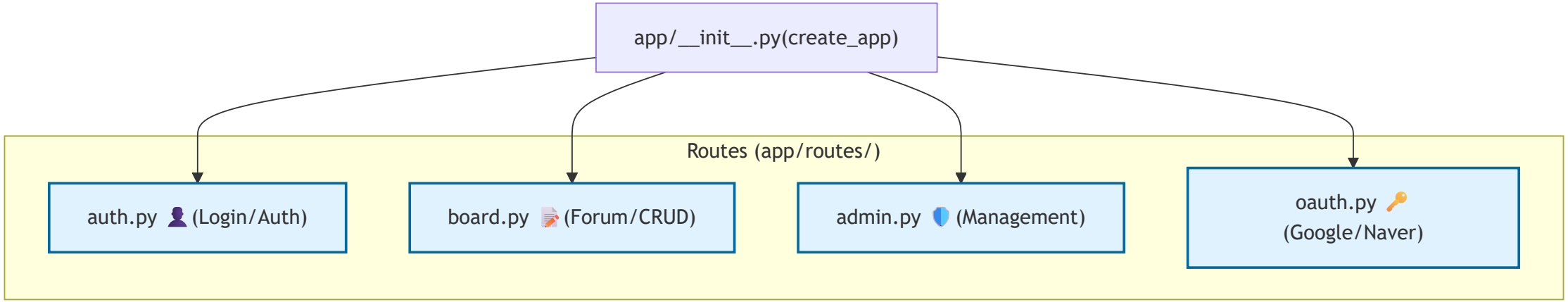
10. Jinja2: SSR(Server-Side Rendering)

- **Inheritance:** `base.html` 하나만 고치면 로그인, 게시판, 메인 모든 페이지의 디자인이 한 번에 바뀝니다.
- **Dynamic Content:** `{{ ... }}` 문법을 통해 서버의 데이터를 HTML 사이사이에 안전하게 끼워 넣습니다.
- **Logic in Template:** `{% if ... %}` 를 통해 로그인 여부에 따라 메뉴를 다르게 보여줍니다.

11. Flask Blueprints: 모듈화의 정석

- 모든 코드를 `app.py` 에 적는 것은 재앙입니다.
- 기능별로 폴더와 파일을 나눠 병렬 개발이 가능하게 설계했습니다 (`app/routes/`).

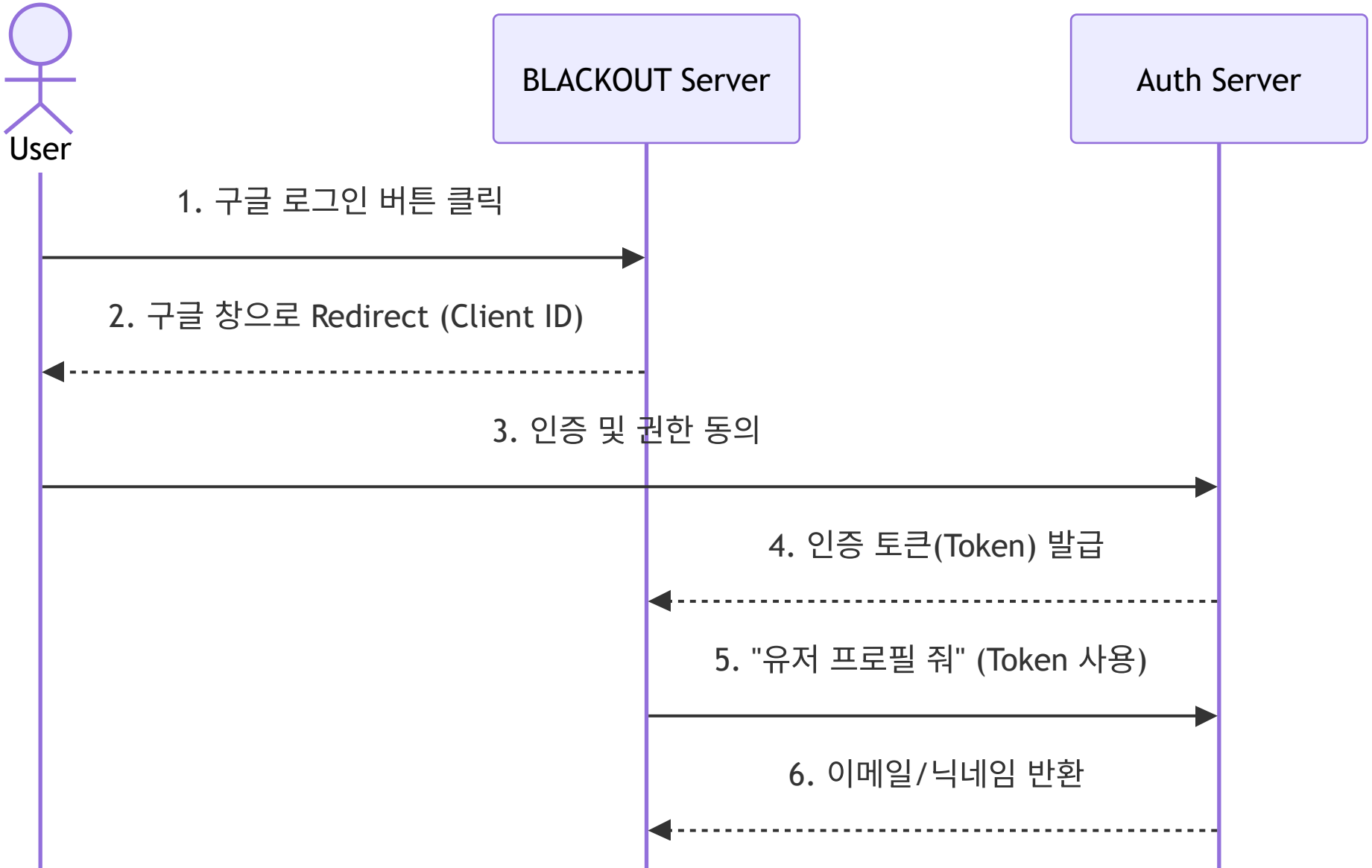
[Diagram] 백엔드 라우팅 모듈화 구조



12. OAuth 2.0: 신뢰 기반의 인증

- 직접 로그인 정보를 저장하는 대신, Google의 보안망을 빌려 쓰는 방식입니다.
- `Flask-Dance` 라이브러리를 통해 복잡한 토큰 교환 알고리즘을 단 몇 줄로 구현했습니다.

[Diagram] OAuth 2.0 인증 시퀀스 흐름



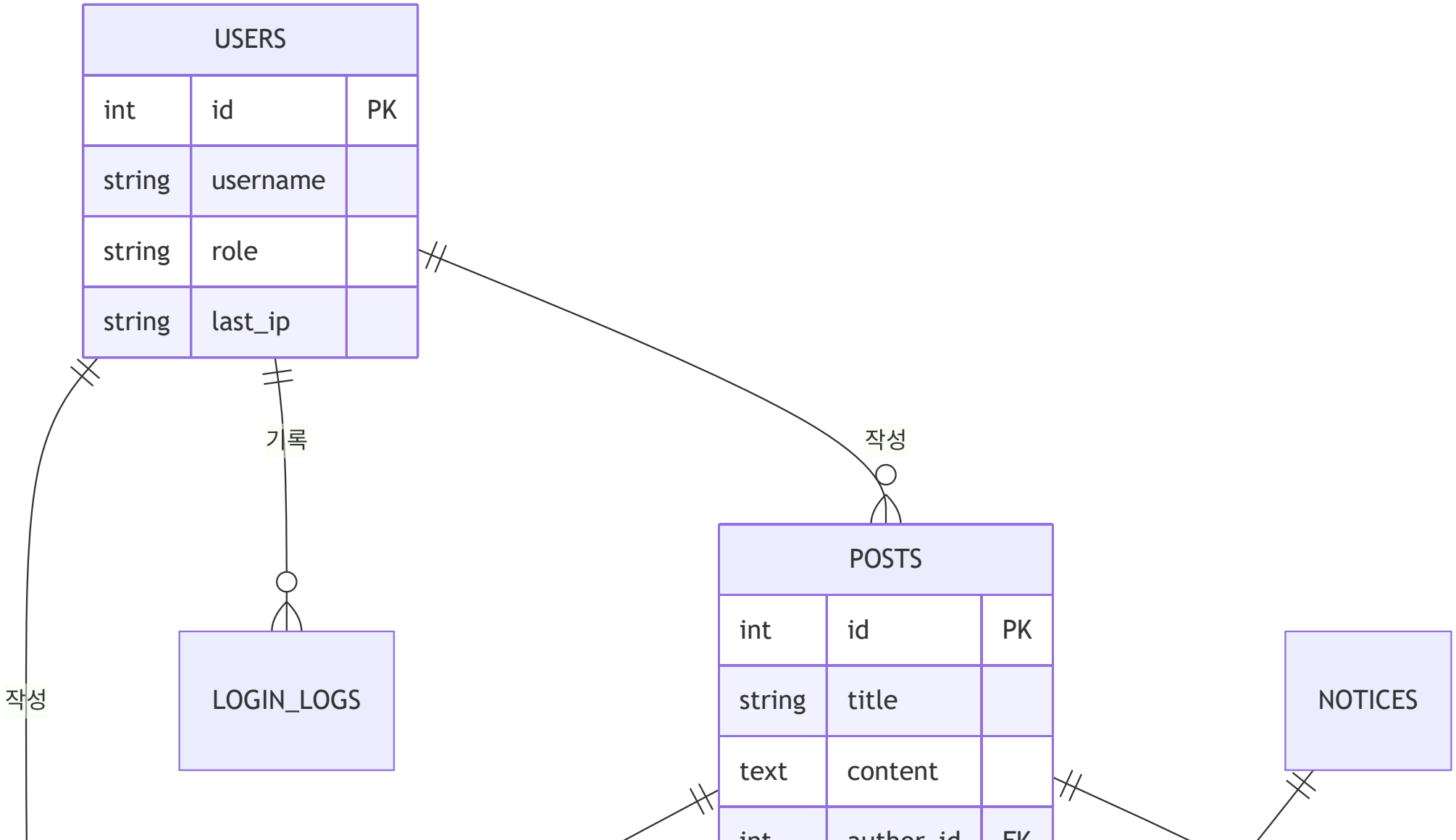
13. Part 4: 데이터 요새 (Database Layer)

- "데이터는 애플리케이션의 기억입니다."
- MariaDB를 통해 데이터를 어떻게 구조화하고 저장하는지 알아봅니다.

14. 8-Table 관계형 데이터 모델 (ERD)

- 게시글, 댓글, 파일, 로그 등 모든 데이터가 거미줄처럼 유기적으로 연결되어 있습니다.

[Diagram] 데이터베이스 개체 관계도 (ERD)



15. Raw SQL vs ORM

- 편한 ORM(SQLAlchemy) 대신 **Raw SQL(PyMySQL)**을 선택한 이유:
- SQL의 본질을 이해하고, 어떻게 해킹 공격이 SQL 문장을 망가뜨리는지 직접 체험하기 위함입니다.
- `database.py`에서 커넥션 풀을 관리하며 효율적인 자원 할당을 수행합니다.

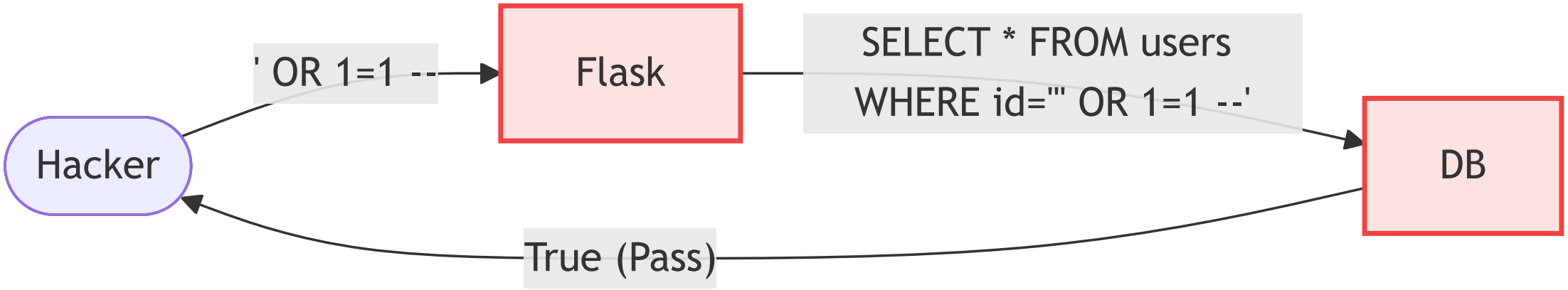
16. Part 5: Security Lab (Vulnerability Targets)

- "공격자의 눈을 가져야 방어자의 손을 완성할 수 있습니다."
- 본 프로젝트에 심어진 의도적 취약점들을 하나씩 분석합니다.

17. [VULN] SQL Injection (SQLi)

- **원인**: 유저 입력값을 정제 없이 쿼리 문장에 직접 합쳤을 때 발생합니다.
- **공격**: `' OR 1=1 --` 입력을 통해 모든 계정의 비밀번호를 우회합니다.
- **방어**: `%s` 문법(Parameterization)을 사용해 입력값을 데이터로만 취급해야 합니다.

[Diagram] SQL Injection 공격 메커니즘



18. [VULN] 크로스 사이트 스크립팅 (XSS)

- **원인**: 유저가 올린 게시글 내용이 필터 없이 타인의 브라우저에서 실행될 때 발생합니다.
- **공격**: `<script>fetch('...cookie')</script>`
- **방어**: `escape()` 함수나 Jinja2의 기본 자동 필터링을 활성화해야 합니다.

19. [VULN] SSRF (Server-Side Request Forgery)

- **원인**: 서버가 유저가 준 URL로 대신 요청을 보내게 설계된 환경에서 발생합니다.
- **공격**: `http://localhost:3306` 으로 요청을 보내 내부 DB를 직접 타격.
- **방어**: 허용된 도메인(Whitelist) 외에 요청을 보내지 못하도록 차단합니다.

20. [VULN] IDOR & Command Injection

- **IDOR:** `post/edit/25` 번호를 `26` 으로 고쳐서 남의 글을 고치는 권한 부족 현상.
- **Cmd Injection:** `os.system('ping ' + target)` 에 `; rm -rf /` 를 섞어 서버 전체를 파괴하는 공격.

21. Part 6: 인프라 요새 (Dockerizing)

- 더 이상 내 컴퓨터의 설정에 얽매이지 않습니다. 어디서든 동일하게 작동하는 환경을 구축합니다.

22. Dockerfile: 서비스의 레시피

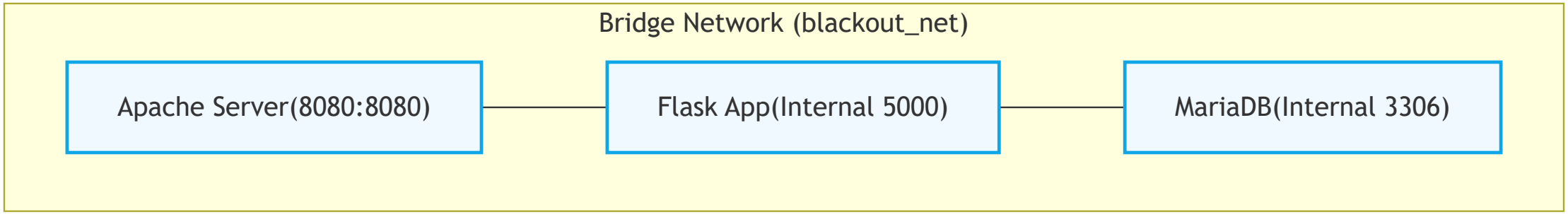
- `python:3.9-slim` 베이스 위에 필요한 라이브러리를 하나씩 쌓아 올립니다.

```
# 1. 얇은 리눅스 위에 파이썬 설치
FROM python:3.9-slim
# 2. MariaDB 클라이언트 등 필수 패키지 설치
RUN apt-get update && apt-get install -y gcc default-libmysqlclient-dev
# 3. 내 소스코드와 라이브러리 목록 복사
COPY . .
# 4. Gunicorn 서비스 실행 (멀티 프로세스)
CMD ["gunicorn", "--workers", "4", "run:app"]
```

23. Docker Compose: 오케스트레이션

- 3개의 컨테이너(Apache, Flask, MariaDB)를 하나의 유닛(Unit)으로 관리합니다.
- **Networks:** 컨테이너끼리만 통신하게 하여 외부 직접 공격을 차단합니다.
- **Volumes:** 컨테이너를 껏다 켜도 내 소중한 DB와 업로드 파일이 유지되게 합니다.

[Diagram] 도커 네트워크 및 서비스 구조



24. Gunicorn & WSGI: Production Hardening

- Flask 내장 서버는 개발용입니다. 실제 서비스에선 **Gunicorn**이 필요합니다.
- Gunicorn은 4개의 워커(Worker)를 두어, 한 명이 바빠도 다른 워커가 즉시 응답을 처리합니다.
- **Failover:** 워커 하나가 에러로 죽어도 메인 매니저가 즉시 새 워커를 '부활'시킵니다.

25. 마무리: 엔지니어링의 본질

1. **격리:** 인프라는 도커로, 로직은 블루프린트로 격리하여 관리합니다.
2. **검증:** 모든 입력값은 독극물로 취급하며 철저히 거릅니다.
3. **자동화:** 레시피(IaC)만 있다면 신입 사원도 1초 만에 서버를 구축할 수 있어야 합니다.

Q&A: BLACKOUT 을 마무리하며

이제 여러분은 단순 코더를 넘어, 전체 시스템을 설계할 수 있는 아키텍트의 길을 걷기 시작했습니다.

> **감사합니다**. 발표를 마치고 질의응답을 시작합니다.